Laboratory #8: CNN Inference on Network Packets with NVIDIA DOCA
Instructor: Young H. Cho, Ph.D.

Objectives

This lab should be done as a GROUP.
By the end of this lab, you will be able to:

1. Convert a PyTorch CNN to TensorRT and run inference on data already in GPU memory.
2. Compare PyTorch FP32, TensorRT FP32, and TensorRT FP16 inference performance.
3. Send images as UDP packets and record them in a PCAP file.
4. Receive the packets with DPDK and rebuild the images.
5. Classify the received images on the GPU.
6. Build and run an NVIDIA DOCA GPUNetIO sample in NVIDIA's DOCA container.
7. Run GPU-initiated RDMA between two GPUs with DOCA GPUNetIO on CloudLab, and compare it with CPU-initiated RDMA.

With DOCA GPUNetIO, an NVIDIA ConnectX or BlueField network card writes packet data directly into GPU memory, and CUDA kernels receive the packets. DOCA GPUNetIO overview: https://networking-docs.nvidia.com/doca/archive/3-5-0/doca-gpunetio
Parts 1 to 6 run on a GPU instance that you create and pay for on a Cloud, as in Laboratory #6. Part 7 needs the network card and runs on CloudLab in a time slot reserved for your team.

Part 1: Setting Up the Environment

Step 1.1: Create a GPU Instance

Create a Linux virtual machine with an NVIDIA GPU on a Cloud. Part 6 runs Docker, so you need a full virtual machine, not a GPU container. On Google Cloud we suggest:

- GPU: NVIDIA L4 (G2 machine type, such as g2-standard-4). A T4 from Laboratory #6 also works for Parts 1 to 6.
- OS: Ubuntu 22.04 LTS
- Boot disk: 100 GB, because the DOCA container in Part 6 is large

Install the NVIDIA driver with Google's guide: https://cloud.google.com/compute/docs/gpus/install-drivers-gpu. Use driver 580 or newer, because the default pip builds of PyTorch and TensorRT use CUDA 13, which requires it. Stop the instance whenever you are not using it, because you pay for every hour it runs.
CUDA, TensorRT, and DPDK do not run on macOS, so Mac users, including M1 and other Apple Silicon Macs, must use the Cloud instance. On Windows with an NVIDIA GPU, you can do Parts 1 to 5 on your own machine under WSL2: https://docs.nvidia.com/cuda/wsl-user-guide/index.html

Step 1.2: Install Packages

```
sudo apt-get update
sudo apt-get install -y build-essential pkg-config git python3-venv \
  pciutils dpdk dpdk-dev libdpdk-dev libpcap-dev \
  rdma-core libibverbs-dev ibverbs-utils
```

Install the CUDA Toolkit by following https://developer.nvidia.com/cuda-downloads. Install only the cuda-toolkit-12-6 package, not cuda, because cuda also installs a driver that can replace the one from Step 1.1. Add the toolkit to your PATH with export PATH=/usr/local/cuda/bin:$PATH, and put that line in ~/.bashrc so nvcc is found in every terminal. Then set up Python:

```
python3 -m venv ~/venv
source ~/venv/bin/activate
pip install torch torchvision onnx onnxscript tensorrt scapy numpy
```

Run source ~/venv/bin/activate again in every new terminal.

Step 1.3: Check

```
nvidia-smi
nvcc --version
python3 -c "import torch; print(torch.cuda.is_available())"
python3 -c "import tensorrt; print(tensorrt.__version__)"
```

Record the GPU, driver, CUDA, PyTorch, and TensorRT versions in your report.

Step 1.4: Troubleshooting Tips

- Could not get lock /var/lib/dpkg/lock-frontend: Ubuntu's automatic updater is running. Wait for it to finish, or run sudo systemctl stop unattended-upgrades, then install again.
- nvidia-smi: command not found: the NVIDIA driver is not installed. Install it with Google's guide from Step 1.1.
- No module named 'torch': activate the virtual environment first.

Part 2: TensorRT Inference from GPU Memory

With GPUNetIO, the image is already in GPU memory when inference starts, so your inference code must take a GPU buffer. Use ResNet-18 from torchvision.

Step 2.1: Export to ONNX

Export the model once in FP32 and once in FP16, with a dynamic batch dimension.
Example Code:

```
import torch, torchvision
model = torchvision.models.resnet18(weights="DEFAULT").eval().cuda()
x = torch.randn(1, 3, 224, 224, device="cuda")
torch.onnx.export(model, x, "resnet18_fp32.onnx",
    input_names=["input"], output_names=["logits"],
    dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
    opset_version=18)
```

Recent PyTorch versions use the dynamo exporter by default. It writes the weights to a separate .onnx.data file next to the .onnx file. Keep the two files in the same folder, and in Step 2.2 load the model with parser.parse_from_file(), which finds the .data file from the model's path. If you call parser.parse() on the raw bytes instead, pass the model's path as its second argument so TensorRT can find the .data file. The exporter also warns that dynamic_axes is not recommended with dynamo and suggests dynamic_shapes; for ResNet-18 the export still succeeds. Write the FP16 export yourself. Reference: https://docs.pytorch.org/docs/stable/onnx.html

Step 2.2: Build TensorRT Engines

TensorRT 11 builds strongly typed networks and removed the per-precision builder flags such as BuilderFlag.FP16, so the FP16 engine comes from the FP16 ONNX file. The STRONGLY_TYPED network flag is no longer needed. BuilderFlag.TF32 is still on by default, so an FP32 engine on an Ampere or newer GPU may run some math in TF32; keep this in mind when you compare FP32 logits. Migration guide: https://docs.nvidia.com/deeplearning/tensorrt/latest/api/migration-guide.html
Example Code:

```
import tensorrt as trt
logger = trt.Logger(trt.Logger.WARNING)
builder = trt.Builder(logger)
# TensorRT 11 networks are always strongly typed
network = builder.create_network()
parser = trt.OnnxParser(network, logger)
parser.parse_from_file("resnet18_fp32.onnx")
config = builder.create_builder_config()
# TODO: add an optimization profile for batch sizes 1 to 32
engine = builder.build_serialized_network(network, config)
open("resnet18_fp32.engine", "wb").write(engine)
```

Complete the optimization profile using the TensorRT documentation: https://docs.nvidia.com/deeplearning/tensorrt/latest/index.html

Step 2.3: Run Inference from GPU Buffers

Allocate the input and output buffers in GPU memory and give their addresses to TensorRT:
Example Code:

```
runtime = trt.Runtime(logger)
engine = runtime.deserialize_cuda_engine(open("resnet18_fp32.engine", "rb").read())
ctx = engine.create_execution_context()
ctx.set_input_shape("input", (batch, 3, 224, 224))
ctx.set_tensor_address("input", inp.data_ptr())    # inp: CUDA tensor
ctx.set_tensor_address("logits", out.data_ptr())   # out: CUDA tensor
ctx.execute_async_v3(stream.cuda_stream)
```

The buffer data types must match engine.get_tensor_dtype(). execute_async_v3 only queues the work on a CUDA stream, so make sure the input is ready before inference and the inference has finished before you read the output. Python API: https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/python-api-docs.html

Step 2.4: Measure

Time each implementation for batch sizes 1, 8, and 32 with warm-up runs, and average over many runs. Random input is enough for timing. For each engine, also record the largest absolute difference in the logits, and the top-1 agreement with PyTorch on the same input.

Implementation	Batch 1	Batch 8	Batch 32
PyTorch FP32	X ms	Y ms	Z ms
TensorRT FP32	A ms	B ms	C ms
TensorRT FP16	D ms	E ms	F ms

Compute speedup = PyTorch time / TensorRT time, and throughput in images per second.

Part 3: Images as Network Packets

A 224 x 224 RGB image is 150,528 bytes, so it must be split across many UDP packets. A PCAP file records packets, and DPDK can read it as if the packets arrived on a network card.

Step 3.1: Packet Format

Each UDP payload is a header followed by one chunk of image data. Include at least these fields:

Field	Size	Meaning
magic	4 bytes	Constant that marks your packets
frame_id	4 bytes	Image this chunk belongs to
seq	2 bytes	Chunk number within the image
nseg	2 bytes	Total chunks in the image

offset	4 bytes	Byte offset of the chunk in the image
length	2 bytes	Image bytes in this packet
frame_bytes	4 bytes	Total image size

With 1,400 bytes of image data per packet, how many packets does one image need? Check that each packet fits in a 1,500-byte MTU.

Step 3.2: Generate the PCAP

Write a Python program that creates 16 test images, splits them into packets, and writes images.pcap. Save the original images so you can check the receiver later. Scapy builds packets by stacking layers:
Example Code:

```
pkt = Ether(src="02:00:00:00:00:01", dst="02:00:00:00:00:02") / \
      IP(src="10.0.0.1", dst="10.0.0.2") / \
      UDP(sport=40000, dport=5000) / Raw(payload)
wrpcap("images.pcap", packets)
```

Add an option that shuffles the packet order. Scapy: https://scapy.readthedocs.io/en/latest/usage.html

Part 4: Receiving Packets with DPDK

DPDK is a set of libraries for packet processing in user space. Its pcap driver turns a PCAP file into a virtual port, so no special hardware is needed. Getting started: https://doc.dpdk.org/guides/linux_gsg/index.html. Pcap driver: https://doc.dpdk.org/guides/nics/pcap.html

Step 4.1: Write the Receiver

Example Code:

```
#include <rte_eal.h>
#include <rte_ethdev.h>
#include <rte_mbuf.h>

int main(int argc, char **argv) {
  if (rte_eal_init(argc, argv) < 0)
    rte_exit(EXIT_FAILURE, "EAL init failed\n");
  struct rte_mempool *pool = rte_pktmbuf_pool_create("mbufs", 8191,
      256, 0, RTE_MBUF_DEFAULT_BUF_SIZE, rte_socket_id());
  struct rte_eth_conf conf = {0};
  rte_eth_dev_configure(0, 1, 1, &conf);   // port 0: 1 RX, 1 TX queue
  rte_eth_rx_queue_setup(0, 0, 1024, rte_socket_id(), NULL, pool);
  rte_eth_tx_queue_setup(0, 0, 1024, rte_socket_id(), NULL);
  rte_eth_dev_start(0);

  struct rte_mbuf *bufs[32];
  for (;;) {
    uint16_t n = rte_eth_rx_burst(0, 0, bufs, 32);
    for (uint16_t i = 0; i < n; i++) {
      uint8_t *pkt = rte_pktmbuf_mtod(bufs[i], uint8_t *);
      // TODO: check the Ethernet, IPv4 and UDP headers,
      //       then copy the chunk into its image
      rte_pktmbuf_free(bufs[i]);
    }
    // TODO: stop when the whole PCAP file has been read
  }
}
```

Complete the receiver. It must check the Ethernet, IPv4, and UDP headers, ignore packets for other ports, rebuild each image in any arrival order, ignore duplicate chunks, and write the complete images to a file. Network headers are big-endian. The pcap port does not report the end of the file, so decide how your program knows it is done.

Step 4.2: Compile and Run

```
gcc -O2 -Wall $(pkg-config --cflags libdpdk) dpdk_rx.c -o dpdk_rx \
  $(pkg-config --libs libdpdk)
sudo ./dpdk_rx -l 0 --no-huge -m 512 --no-pci \
  --vdev 'net_pcap0,rx_pcap=images.pcap,tx_pcap=/dev/null'
```

-l 0 runs on CPU core 0, --no-huge -m 512 uses 512 MB of normal memory instead of hugepages, --no-pci skips PCI devices, and --vdev creates port 0 from the PCAP file. Arguments after a -- separator, such as -- frames_rx.bin for the output file, go to your program. rte_eal_init returns the number of arguments it used (ret). After it returns, argv[ret] holds the program name, so your first argument is argv[ret + 1]. Check that every image is rebuilt byte for byte, with packets in order and shuffled. Record the packets received and dropped, and the complete images.

Part 5: Classifying the Received Images

Copy the rebuilt images to GPU memory and classify them with your FP16 engine. Do the preprocessing on the GPU: convert the height x width x channel bytes to the channel-first float layout and normalize them the way the pretrained weights expect. torchvision bundles these transforms with the weights: https://docs.pytorch.org/vision/stable/models.html
Check that each received image gets the same class as its original. Measure the CPU-to-GPU copy time with pinned memory. This is the copy GPUNetIO removes.

Part 6: NVIDIA DOCA Container

DOCA is NVIDIA's SDK for its ConnectX network cards and BlueField DPUs. NVIDIA publishes it as a container: https://catalog.ngc.nvidia.com/orgs/nvidia/teams/doca/containers/doca

Step 6.1: Docker with GPU Access

Install Docker and the NVIDIA Container Toolkit: https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html. Check GPU access:

```
sudo docker run --rm --gpus all ubuntu nvidia-smi -L
```

Step 6.2: Explore DOCA

```
IMG=nvcr.io/nvidia/doca/doca:devel-cuda13.0.0-3.5.0-devel-host
sudo docker pull $IMG
sudo docker run --rm -it --gpus all --privileged --net=host $IMG bash
# inside the container:
doca_caps --version
doca_caps --list-libs
doca_caps --list-devs
```

This image needs NVIDIA driver 580 or newer: https://docs.nvidia.com/deploy/cuda-compatibility/index.html. DOCA 3.5 has no CUDA 12 image, so update an older driver. Record the installed DOCA libraries and the devices DOCA finds.

Step 6.3: GPUNetIO Receive Sample

gpunetio_simple_receive receives UDP packets into GPU memory with a CUDA kernel. Before building, add your GPU's architecture to the sample's meson.build file, in both the compile and the link flag lists. sm_75 (T4) and sm_80 (A30) are already listed, but sm_89 for the L4 is not. Sample guide: https://networking-docs.nvidia.com/doca/archive/3-5-0/gpunetio-sample-guide

```
cd /opt/mellanox/doca/samples/doca_gpunetio/gpunetio_simple_receive
meson setup /tmp/build && ninja -C /tmp/build
/tmp/build/doca_gpunetio_simple_receive \
  -n <NIC PCI address> -g <GPU PCI address> -e 0
```

Get the GPU address with nvidia-smi --query-gpu=pci.bus_id --format=csv and pass the short form, such as 00:04.0. Get the NIC address from lspci. -e sets how the CUDA kernel receives: 0 per thread, 1 per warp, 2 per block (the default). If meson cannot find DOCA or CUDA, set PATH, PKG_CONFIG_PATH, and LD_LIBRARY_PATH as shown in the sample guide. On a Cloud instance, expect the sample to stop at init_doca_device, because DOCA finds no ConnectX or BlueField device at that address. Explain why the sample stops, using the requirements in the installation guide: https://networking-docs.nvidia.com/doca/archive/3-5-0/gpunetio-installation-and-setup

Also build the open-source GPUNetIO on GitHub and run one of its examples: https://github.com/NVIDIA-DOCA/gpunetio. Build the library and its examples with make install install_examples PREFIX=$HOME/gpunetio/install CUDA_ARCH=89 (89 for the L4, 75 for a T4; the default is 80), then add $HOME/gpunetio/install/lib to LD_LIBRARY_PATH so the examples can find the library. Run one example, such as $HOME/gpunetio/install/examples/gpunetio_verbs_write_lat -g <GPU PCI address> -d mlx5_0. Logging is off by default, so run the example with DOCA_GPUNETIO_LOG=6. Errors about libgdrapi.so.2 (GDRCopy) appear first and are not the cause.

On a Cloud instance, expect it to stop in open_ib_device() with Failed to get RDMA devices list. The open-source version only covers the GPU data path for RDMA Verbs, and every example needs a client and a server. It has no Ethernet data path, so it cannot receive UDP packets. Part 7 runs its RDMA examples between two CloudLab servers.
NOTE: When you finish Part 6, stop or delete your Cloud instance so you are not charged for it.

Part 7: GPU-Initiated RDMA with GPUNetIO on CloudLab

CloudLab provides bare-metal servers for research and education: https://www.cloudlab.us. Part 7 uses two d7525 servers on the Wisconsin cluster. CloudLab's hardware page lists an NVIDIA A30 GPU and a dual-port ConnectX-6 Dx network card for this node type: https://docs.cloudlab.us/hardware.html. The two nodes we tested had the A30, but the card wired to the experiment network was a single-port ConnectX-6, not a ConnectX-6 Dx. One of them also had a BlueField-2 DPU with no cables on its ports. Step 7.4 shows how to check your own nodes.

In Part 7, a CUDA kernel on one server writes into the GPU memory of the other server over RDMA, using the open-source GPUNetIO from Step 6.3. The network card reads and writes GPU memory directly, so the data never passes through CPU memory. You will compare this with RDMA driven by the CPU, and you will also try the UDP receive sample from Step 6.3 and explain what happens on your nodes.

CloudLab has only a few of these servers, so each team gets one reserved 4-hour time slot. One member of each team joins the class project and books the slot with the TA. That member starts the experiment and can give teammates SSH access to the servers.

Step 7.1: Join the Class Project

Create a CloudLab account, add your SSH public key, and request to join the project DOCA-GPU: open https://www.cloudlab.us/signup.php?pid=DOCA-GPU and check that the project field shows DOCA-GPU. Then email the TA your team number and CloudLab username so your request can be approved. Account guide: https://docs.cloudlab.us/users.html

Step 7.2: Create Your Profile and Start the Experiment

Write a CloudLab profile, profile.py, that requests two bare-metal d7525 nodes running Ubuntu 24.04 (image UBUNTU24-64-STD) on one LAN, with node0 at 10.10.1.1 and node1 at 10.10.1.2 (netmask 255.255.255.0). Each node also needs a /mydata file system with room for Docker's data, which Step 7.3 moves there. Start from the geni-lib examples: https://docs.cloudlab.us/geni-lib.html
Put profile.py alone in a new public GitHub repository and create the profile from it in the DOCA-GPU project. CloudLab builds a profile from a repository only if the repository is public, so do not use the one that holds your lab code. Profile guide: https://docs.cloudlab.us/creating-profiles.html
Start an experiment from your profile on the Wisconsin cluster, in your team's group. A reservation holds the nodes for your slot but does not start an experiment, so schedule yours to start when your slot begins.

Step 7.3: Set Up Both Nodes

Set up node0 and node1 for GPUNetIO with the installation guide from Step 6.3. DOCA itself runs in the container, so we did not install DOCA on the hosts. On our nodes, the setup needed:

- The NVIDIA open kernel module driver from the 580 branch and the CUDA 13.0 toolkit: https://docs.nvidia.com/datacenter/tesla/driver-installation-guide/ubuntu.html. The DOCA image needs driver 580 or newer (Step 6.2), and GPUNetIO's dmabuf path needs the open kernel module. Reboot if nvidia-smi fails after the install.
- build-essential, git, linux-headers-$(uname -r), rdma-core, ibverbs-utils, and libibverbs-dev from apt.
- GDRCopy v2.5.2 built in /opt/mellanox/gdrcopy, with its gdrdrv module loaded by its insmod.sh script: https://github.com/NVIDIA/gdrcopy. The commands later in Part 7 expect this path.
- 2,048 hugepages of 2 MB for gpunetio_simple_receive in Step 7.5, and GPU persistence mode (nvidia-smi -pm 1).
- ACS turned off on the PCIe bridges above the GPU and the network card. You cannot reach the BIOS on CloudLab, so use the setpci method from the installation guide.
- Docker and the NVIDIA Container Toolkit from Step 6.1, with Docker's data-root on /mydata, because the root disk is 64 GB and the DOCA image from Step 6.2 is about 22 GB.

The hugepages, the gdrdrv module, persistence mode, and the ACS settings are lost when a node reboots, so set them again after every reboot. Do not flash the network card's firmware or change its settings with mlxconfig, because the change stays on the node after your experiment ends.

Step 7.4: Check the Hardware

Check each node against the GPUNetIO installation guide from Step 6.3. Find the network interface with the 10.10.1.x address, its PCI address and firmware, the RDMA devices, the NVIDIA network cards, the kernel version, and the PCIe path between the GPU and the network card:

```
ip -br addr
ethtool -i <interface>
ibv_devices
lspci | grep -i mellanox
uname -r
nvidia-smi topo -m
```

On our nodes the 10.10.1.x interface was enp65s0np0, a ConnectX-6 at PCI address 41:00.0 with firmware 20.41, and the GPU was at 25:00.0. nvidia-smi topo -m reported NODE between the GPU and the card, which means the path between them crosses the link between two PCIe host bridges in the same NUMA node. The installation guide recommends PIX or PXB and says to avoid NODE and SYS. Compare the results with your Cloud instance in Part 6.

Step 7.5: Try the UDP Receive Sample

On node0, run gpunetio_simple_receive in the DOCA container with the PCI addresses of the 10.10.1.1 interface and the GPU. sm_80 for the A30 is already in meson.build. The container needs the RDMA

devices, the hugepages, and the GDRCopy library from the host. The example below mounts the GDRCopy library you built in Step 7.3.

Example Code:

```
IMG=nvcr.io/nvidia/doca/doca:devel-cuda13.0.0-3.5.0-devel-host
sudo docker run --rm -it --privileged --net=host --gpus all \
  -v /dev/infiniband:/dev/infiniband \
  -v /dev/hugepages:/dev/hugepages \
  -v /opt/mellanox/gdrcopy/src:/opt/gdrcopy:ro $IMG bash
# inside the container:
export LD_LIBRARY_PATH=/opt/gdrcopy:$LD_LIBRARY_PATH
cd /opt/mellanox/doca/samples/doca_gpunetio/gpunetio_simple_receive
meson setup /tmp/build && ninja -C /tmp/build
/tmp/build/doca_gpunetio_simple_receive -n 41:00.0 -g 25:00.0 -e 0
```

On the nodes we tested, the sample stopped while it created the DOCA Flow port, with Failed to get hws cap. The sample steers the UDP packets into its GPU receive queue with DOCA Flow hardware steering, and the ConnectX-6 firmware rejects the capability query for it. Look up which network cards the installation guide recommends, and explain the result in your report. The rest of Part 7 uses RDMA, which this card supports.

Step 7.6: Turn Off the IOMMU

We ran the RDMA tests with the IOMMU off (see the IOMMU notes in the installation guide). On both nodes:

```
sudo sed -i 's/^GRUB_CMDLINE_LINUX_DEFAULT="/&amd_iommu=off iommu=off /'
/etc/default/grub
grep GRUB_CMDLINE_LINUX_DEFAULT /etc/default/grub
sudo update-grub && sudo reboot
```

After the reboot, check that cat /proc/cmdline ends with amd_iommu=off iommu=off. Set the hugepages, the gdrdrv module, persistence mode, and the ACS settings again, as in Step 7.3. Then run ibv_devices again, because the RDMA device can come back with a new name: on our nodes mlx5_0 became rocep65s0. Use the current name wherever the commands below say rocep65s0.

Step 7.7: Build the Open-Source GPUNetIO

On both nodes, build the library and its examples from Step 6.3 for the A30. CUDA_ARCH=80 is the default, so no change is needed. The examples need the GPUNetIO library and GDRCopy at run time, so add both to LD_LIBRARY_PATH, and put the two export lines in ~/.bashrc:

```
git clone https://github.com/NVIDIA-DOCA/gpunetio ~/gpunetio
cd ~/gpunetio
make install install_examples PREFIX=$HOME/gpunetio/install CUDA_ARCH=80
export
LD_LIBRARY_PATH=$HOME/gpunetio/install/lib:/opt/mellanox/gdrcopy/src:$LD_LIBRARY_PAT
H
export EX=$HOME/gpunetio/install/examples
```

If an example still reports that libdoca_gpunetio_host.so cannot be found, find the file under ~/gpunetio/install and add its folder to LD_LIBRARY_PATH.

Step 7.8: GPU-to-GPU Latency

gpunetio_verbs_write_lat is a GPU version of ib_write_lat. On both nodes a CUDA kernel posts RDMA writes into the other node's GPU memory and waits for the reply. Start the server on node0 first, then the client on node1. -g is the GPU address, -d is the RDMA device, -l 3 selects GID index 3 (RoCE v2 with the node's IPv4 address), and -p 1 selects the CPU proxy NIC handler.

node0:

```
DOCA_GPUNETIO_LOG=6 $EX/gpunetio_verbs_write_lat -g 25:00.0 -d rocep65s0 -l 3 -p 1
```

node1:

```
DOCA_GPUNETIO_LOG=6 $EX/gpunetio_verbs_write_lat -g 25:00.0 -d rocep65s0 -l 3 -p 1 -
c 10.10.1.1
```

With the CPU proxy, a CPU thread rings the network card's doorbell for the GPU, while the CUDA kernel still builds the requests and the card moves the data from GPU memory. On our nodes, -p 2 (the GPU rings the doorbell itself) failed with Can't export UAR to GPU after cudaHostRegister returned error 800, and the default -p 0 (AUTO) hung after the kernel started. AUTO falls back to the CPU proxy when the doorbell cannot be mapped into GPU memory (the log says Enabling CPU proxy mode), but the examples start their proxy thread only when you pass -p 1, so with AUTO nothing rings the doorbell. Run -p 2 once, record the error, and explain in your report what the GPU would need in order to ring the doorbell. Record the one-way latency for each message size.

Step 7.9: GPU-to-GPU Bandwidth

gpunetio_verbs_put_bw and gpunetio_verbs_write_bw are GPU versions of ib_write_bw. The CUDA kernel on the client sends. When the client finishes, stop the server with Ctrl+C: both servers then check the data they received and print the result. Run each one with the server on node0 and the client on node1:

```
# node0
DOCA_GPUNETIO_LOG=6 $EX/gpunetio_verbs_put_bw -g 25:00.0 -d rocep65s0 -l 3 -p 1
# node1
DOCA_GPUNETIO_LOG=6 $EX/gpunetio_verbs_put_bw -g 25:00.0 -d rocep65s0 -l 3 -p 1 -c
10.10.1.1
# node0
DOCA_GPUNETIO_LOG=6 $EX/gpunetio_verbs_write_bw -g 25:00.0 -d rocep65s0 -p 1
# node1
DOCA_GPUNETIO_LOG=6 $EX/gpunetio_verbs_write_bw -g 25:00.0 -d rocep65s0 -p 1 -c
10.10.1.1
```

gpunetio_verbs_write_bw rejects -l even though its help text lists it, so leave it out; it then uses GID index 0, the node's link-local IPv6 address. Record the bandwidth and message rate for each message size.

Step 7.10: CPU-Initiated RDMA Baseline

Run the same tests with perftest, where the CPU posts each RDMA write from CPU memory. Ubuntu's perftest is built without CUDA, so it cannot use GPU memory. -x 3 selects the same GID index as -l 3 above. --report_gbits makes ib_write_bw report Gbit/s like the GPUNetIO examples; without it, perftest reports MiB/s.

```
sudo apt-get install -y perftest
# node0
ib_write_lat -d rocep65s0 -x 3
# node1
ib_write_lat -d rocep65s0 -x 3 10.10.1.1
# node0
ib_write_bw -d rocep65s0 -x 3 --report_gbits
# node1
ib_write_bw -d rocep65s0 -x 3 --report_gbits 10.10.1.1
```

Fill in the table from gpunetio_verbs_write_lat and gpunetio_verbs_put_bw, which use the same GID index as perftest, and plot the GPUNetIO latency and bandwidth against message size. In the Video Log, show the table and explain the difference between the two columns using where each test keeps its data and which processor tells the network card to send.

Measurement	GPUNetIO (GPU-initiated)	perftest (CPU-initiated)
One-way latency, smallest message (μs)		
Bandwidth at 64 KB (Gbps)		

NOTE: CloudLab erases the servers when the experiment ends. Copy your results off with scp, then terminate the experiment so the next team can use the servers.

Deliverables

1. Source code: ONNX export, engine build, GPU buffer runner, packet generator, DPDK receiver, GPU classification, and your Part 7 command lines and logs in GitHub
2. Video Log: Include the explanation of the following in the video log.
   - Part 2 results table with speedups
   - Image reassembly with packets in order and shuffled
   - DOCA container output and where the sample stops on the Cloud instance
   - What gpunetio_simple_receive does on your d7525 nodes, and your GPU-to-GPU RDMA results compared with CPU-initiated RDMA
   - Analysis of the overhead of moving network data to the GPU